

GEBZE TECHNICAL UNIVERSITY
ELECTRONIC ENGINEERING
ELC 335

MICROPROCESSORS LABORATORY-I

Büşra Ülker
Ayşe Beril Çevik
Fouad Aladhami

1 PROBLEM 1 3

1.1 STM32G031K8 NUCLEO PINOUT 3
1.2 ON BOARD IC'S 3
1.3 PERIPHERALS AND THEIR USAGE 4
1.3.1 GPIOs: 4
1.3.2 SRAM: 4
1.3.3 FLASH: 4
1.3.4 DMA: 4
1.3.5 DMAMUX: 4
1.3.6 CRC: 5
1.3.7 RNG: 5
1.3.8 RCC: 5
1.3.9 USART(universal synchronous asynchronous receiver transmitter): 5
1.3.10 ADC: 5
1.3.11 SPI(internal Standard Peripheral Interface): 5
1.3.12 TIM: 5
1.3.13 I2C: 5
1.3.14 RTC: 5
1.3.15 PLL: 5

2 INTRODUCTION TO PROGRAMMMING REGISTERS 6

2.1 RCC CONFIGURATION I/O PORT CLOCK ENABLE REGISTER (RCC_IOPENR) 6

3 PROBLEM 2 8

4 PROBLEM 3 11

5 PROBLEM 4 14

5.1 CLOCK SETTINGS 14
5.2 GPIO (PA8) CONFIGURATION 14
5.3 TOGGLING LED AND DELAY LOOP 14

1 PROBLEM 1

1.1 Stm32g031k8 nucleo pinout

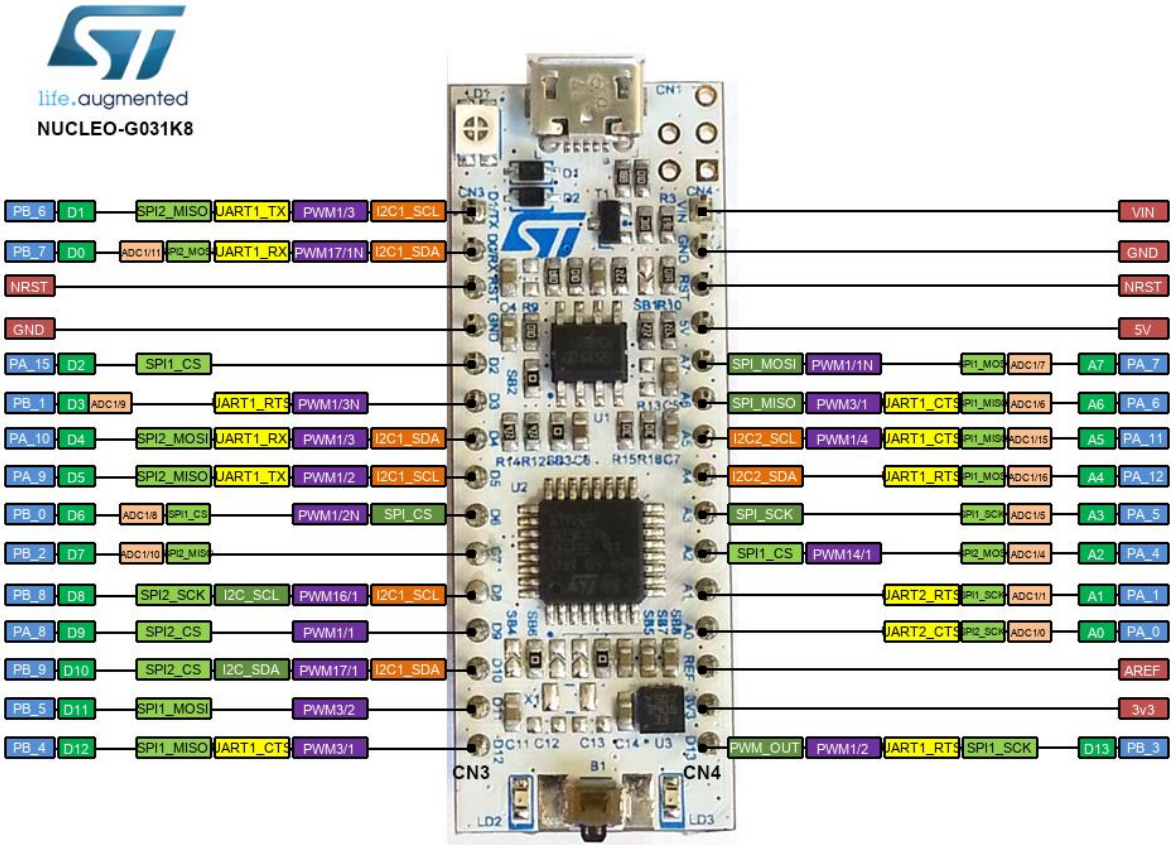


Figure 1:Stm32g031k8 nucleo board pinout

As shown at Figure1, the pins can used for multiple purposes. The only digital pins are labeled with Dx and pins that can read analog signals with ADC is labeled with Ax.

1.2 On board IC's



Figure 2: Stm32g031k8 nucleo board back

The board include a usb port, this port allows code running and debugging using ST- link. For ST-link a stm32f103 IC is used and is connected to the back of the nucelo board. The stm32g0 IC operate at 3.3V because of it a 5V 3.3V regulator appear on the board. A 16Mhz oscillator appear to clock the stm32g0.

1.3 Peripherals and their usage

1.3.1 GPIOs:

GPIOs (general-purpose input/output) port handles both incoming and outgoing digital signals. As an input port, it can be used to communicate to the CPU the ON/OFF signals received from switches, or the digital readings received from sensors.

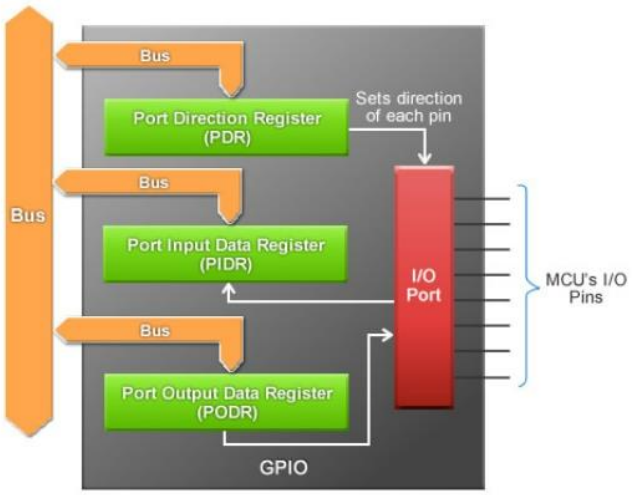


Figure 3: GPIO structure

1.3.2 SRAM:

SRAM (static RAM) is a type of random-access memory (RAM) that retains data bits in its memory if power is being supplied. Unlike dynamic RAM (DRAM), which must be continuously refreshed, SRAM does not have this requirement, resulting in better performance and lower power usage.

1.3.3 FLASH:

The STM32G0 embeds up to 128 Kbytes of single-bank Flash memory. The Flash memory interface manages all memory access (read, programming and erasing) as well as memory protection, security and option bytes.

1.3.4 DMA:

The direct memory access (DMA) controller is a bus master and system peripheral with single-AHB architecture. With 5 channels, it performs data transfers between memory-mapped peripherals and/or memories, to offload the CPU.

1.3.5 DMAMUX:

The DMAMUX request multiplexer enables routing DMA request lines from the peripherals to the DMA controllers in the products.

1.3.6 CRC:

The CRC (cyclic redundancy check) calculation unit is used to get a CRC code using a configurable generator polynomial value and size, perform data integrity checks of off-/on-chip memories as a background task without CPU intervention.

1.3.7 RNG:

Provides random numbers which are used when producing an unpredictable result is desirable.

1.3.8 RCC:

The RCC peripheral is used to control the internal peripherals, as well as the reset signals and clock distribution. The RCC gets several internal (LSI, HSI and CSI) and external (LSE and HSE) clocks. They are used as clock sources for the hardware blocks, either directly or indirectly

1.3.9 USART (universal synchronous asynchronous receiver transmitter):

The USART is a hardware that enables the device to communicate using serial protocol.

1.3.10 ADC:

The analog-to-digital converters allows the microcontroller to accept an analog value like a sensor output and convert the signal into the digital domain

1.3.11 SPI (internal Standard Peripheral Interface):

Provides. simple communication interface allowing the microcontroller. to communicate with external devices.

1.3.12 TIM:

TIM peripheral is a multi-channel timer unit. It's include advanced-control timers, general-purpose timers and basic timers.

1.3.13 I2C:

The I2C is a multi-master, multi-slave, synchronous, bidirectional, half-duplex serial communication bus.

1.3.14 RTC:

RTC, is a digital clock with a primary function to keep accurate track of time even when a power supply is turned off or a device is placed in low power mode.

1.3.15 PLL:

PLL is a clock generation engine in the MCU which is used to generate the clock speed which is much higher than the internal HSI or external clock

2 INTRODUCTION TO PROGRAMMMING Registers

2.1 RCC Configuration I/O port clock enable register (RCC_IOPENR)

The register shown in Figure 1 is defined as RCC_IOPENR, reset and clock setting is activated by making the related bits 1.

5.4.13 I/O port clock enable register (RCC_IOPENR)

Address: 0x34

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	GPIOFEN	GPIOEEN(1)	GPIODEN	GPIOCEN	GPIOBEN	GPIOAEN
										rw	rw	rw	rw	rw	rw

1. Only significant on devices integrating the corresponding peripheral, otherwise reserved. Refer to *Section 1.4: Availability of peripherals.*

Bits 31:6 Reserved, must be kept at reset value.

Bit 5 **GPIOFEN:** I/O port F clock enable
This bit is set and cleared by software.
0: Disable
1: Enable

Bit 4 **GPIOEEN:** I/O port E clock enable(1)
This bit is set and cleared by software.
0: Disable
1: Enable

Bit 3 **GPIODEN:** I/O port D clock enable
This bit is set and cleared by software.
0: Disable
1: Enable

Bit 2 **GPIOCEN:** I/O port C clock enable
This bit is set and cleared by software.
0: Disable
1: Enable

Bit 1 **GPIOBEN:** I/O port B clock enable
This bit is set and cleared by software.
0: Disable
1: Enable

Bit 0 **GPIOAEN:** I/O port A clock enable
This bit is set and cleared by software.
0: Disable
1: Enable

Figure 4: RCC_IOPENR register.

GPIOX base addresses were defined as GPIOX_BASE and these addresses were accessed to the registers that make the settings on the X ports. It is seen that there is an offsett of 0x400 between the ports.

Table 6. STM32G0x1 peripheral register boundary addresses

Bus	Boundary address	Size	Peripheral	Peripheral register map
-	0xE000 0000 - 0xE00F FFFF	1MB	Cortex®-M0+ internal peripherals	-
IOPORT	0x5000 1800 - 0x5FFF FFFF	~256 MB	Reserved	-
	0x5000 1400 - 0x5000 17FF	1 KB	GPIOF	*Section 7.4.12 on page 248*
	0x5000 1000 - 0x5000 13FF	1 KB	GPIOE	*Section 7.4.12 on page 248*
	0x5000 0C00 - 0x5000 0FFF	1 KB	GPIOD	*Section 7.4.12 on page 248*
	0x5000 0800 - 0x5000 0BFF	1 KB	GPIOC	*Section 7.4.12 on page 248*
	0x5000 0400 - 0x5000 07FF	1 KB	GPIOB	*Section 7.4.12 on page 248*
	0x5000 0000 - 0x5000 03FF	1 KB	GPIOA	*Section 7.4.12 on page 248*

Figure 5: GPIOX address map.

Using GPIOX_BASE addresses, the modes of the pins in the MODER registry can be set by accessing the MODER registry with offset 0x00.

7.4.1 GPIO port mode register (GPIOx_MODER) (x =A to F)

Address offset:0x00

Reset value: 0xEBFF FFFF for port A

Reset value: 0xFFFF FFFF for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODE15[1:0]		MODE14[1:0]		MODE13[1:0]		MODE12[1:0]		MODE11[1:0]		MODE10[1:0]		MODE9[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODE7[1:0]		MODE6[1:0]		MODE5[1:0]		MODE4[1:0]		MODE3[1:0]		MODE2[1:0]		MODE1[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **MODE[15:0][1:0]:** Port x configuration I/O pin y (y = 15 to 0)
These bits are written by software to configure the I/O mode.
00: Input mode
01: General purpose output mode
10: Alternate function mode
11: Analog mode (reset state)

Figure 6: GPIOx_MODER Register

Using GPIOX_BASE addresses, the ODR register is accessed with 0x14 Offsett and the pins are powered by assigning a value of 1 to the relevant bits in the ODR register.

7.4.6 GPIO port output data register (GPIOx_ODR) (x = A to F)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OD15	OD14	OD13	OD12	OD11	OD10	OD9	OD8	OD7	OD6	OD5	OD4	OD3	OD2	OD1	OD0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **OD[15:0]:** Port output data I/O pin y (y = 15 to 0)
These bits can be read and written by software.
Note: For atomic bit set/reset, the OD bits can be individually set and/or reset by writing to the GPIOx_BSRR register (x = A..D, F).

Figure 7: GPIOx_ODR register

3 Problem 2

To turn on the LED connected to the PA8 pin, the clock of the GPIOA was first activated by set GPIOAEN bit (bit 0) in RCC_IOPENR register. Then, bits 12-13 were set to 01 in MODER for PA8 to be output. Finally, the LED was powered by setting OD8 to 1 in ODR.

```
/*
 * asm.s
 *
 * author:Fouad Aladhami/Busra Ülker/Ayse Beril Cevik
 *
 * description: Added the necessary stuff for turning on the 4 LEDs on the
G031K8 Nucleo board.
 */

.syntax unified
.cpu cortex-m0plus
.fpu softvfp
.thumb


/* make linker see this */
.global Reset_Handler

/* get these from linker script */
.word _sdata
.word _edata
.word _sbss
.word _ebss
```

```
/* define peripheral addresses from RM0444 page 57, Tables 3-4 */
.equ RCC_BASE,         (0x40021000)          // RCC base address
.equ RCC_IOPENR,       (RCC_BASE   + (0x34)) // RCC IOPENR register offset

.equ GPIOA_BASE,       (0x50000000)          // GPIOA base address
.equ GPIOA_MODER,      (GPIOA_BASE + (0x00)) // GPIOA MODER register offset
.equ GPIOA_ODR,        (GPIOA_BASE + (0x14)) // GPIOA ODR register offset


/* vector table, +1 thumb mode */
.section .vectors
vector_table:
       .word _estack             /*     Stack pointer */
       .word Reset_Handler +1    /*     Reset handler */
       .word Default_Handler +1  /*       NMI handler */
       .word Default_Handler +1  /* HardFault handler */
       /* add rest of them here if needed */

/* reset handler */
.section .text
Reset_Handler:
       /* set stack pointer */
       ldr r0, =_estack
       mov sp, r0

       /* initialize data and bss
        * not necessary for rom only code
        * */
       bl init_data
       /* call main */
       bl main
       /* trap if returned */
       b .
/* initialize data and bss sections */
.section .text
init_data:

       /* copy rom to ram */
       ldr r0, =_sdata
       ldr r1, =_edata
       ldr r2, =_sidata
       movs r3, #0
       b LoopCopyDataInit

       CopyDataInit:
              ldr r4, [r2, r3]
              str r4, [r0, r3]
              adds r3, r3, #4

       LoopCopyDataInit:
              adds r4, r0, r3
              cmp r4, r1
              bcc CopyDataInit

       /* zero bss */
       ldr r2, =_sbss
       ldr r4, =_ebss
       movs r3, #0
       b LoopFillZerobss
```

```
      FillZerobss:
            str  r3, [r2]
            adds r2, r2, #4

      LoopFillZerobss:
            cmp r2, r4
            bcc FillZerobss

      bx lr


/* default handler */
.section .text
Default_Handler:
      b Default_Handler


/* main function */
.section .text
main:
      /* enable GPIOA clock, bit0 on IOPENR */
      ldr r6, =RCC_IOPENR
      ldr r5, [r6]
      movs r4, 0x1
      orrs r5, r5, r4 /*Enable GPIOA clock*/
      str r5, [r6]/*Store the r5 register value to IOPENR register*/

      /* setup PA8 for led 01 for bits 12-13 in MODER*/
      ldr r6, =GPIOA_MODER
      ldr r5, [r6]
      ldr r4, =0x30000
      mvns r4, r4
      ands r5, r5, r4
      ldr r4, =0x10000
      orrs r5, r5, r4
      str r5, [r6]

      /* turn on led connected to A8 in ODR */
      ldr r6, =GPIOA_ODR
      ldr r5, [r6]
      ldr r4,=0x100
      orrs r5, r5, r4
      str r5, [r6]

      /* for(;;); */
      b .

      /* this should never get executed */
      nop
```

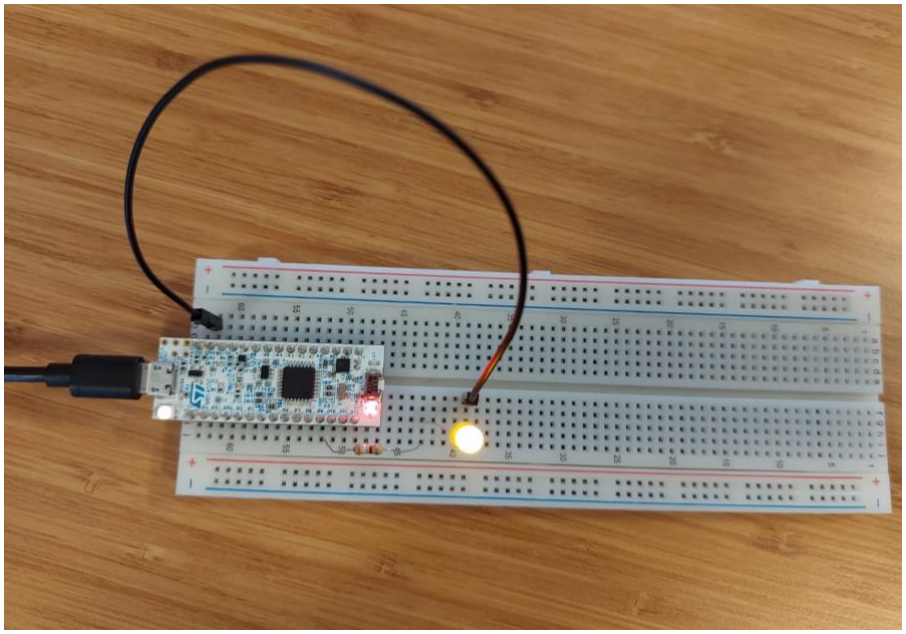


Figure 8: Led on at PA8 pin.

4 PROBLEM 3

To turn on the LED connected to the PA11, PA12, PB4, PB5 pin, the clock setting of GPIOA and GPIOB was first activated. Then, MODE Pin values were set to 01 in MODER to make the pins output. Finally, the relevant pins in ODR were set to 1 and the LED was powered.

```
/*
 * asm.s
 *
 * author:Fouad Aladhami/Büşra Ülker/Ayşe Beri Çevik
 *
 * description: Added the necessary stuff for turning on the LED on the G031K8
Nucleo board.
 */


.syntax unified
.cpu cortex-m0plus
.fpu softvfp
.thumb


/* make linker see this */
.global Reset_Handler

/* get these from linker script */
.word _sdata
.word _edata
.word _sbss
.word _ebss


/* define peripheral addresses from RM0444 page 57, Tables 3-4 */
```

```
.equ RCC_BASE,         (0x40021000)          // RCC base address RCC(Reset and
Clock Control)
.equ RCC_IOPENR,       (RCC_BASE   + (0x34)) // RCC IOPENR register offset

.equ GPIOA_BASE,       (0x50000000)          // GPIOA base address
.equ GPIOA_MODER,      (GPIOA_BASE + (0x00)) // GPIOA MODER register offset
.equ GPIOA_ODR,        (GPIOA_BASE + (0x14)) // GPIOA ODR register offset


.equ GPIOB_BASE,       (0x50000400)          // GPIOB base address
.equ GPIOB_MODER,      (GPIOB_BASE + (0x00)) // GPIOB MODER register offset
.equ GPIOB_ODR,        (GPIOB_BASE + (0x14)) // GPIOB ODR register offset



/* vector table, +1 thumb mode */
.section .vectors
vector_table:
       .word _estack             /*     Stack pointer */
       .word Reset_Handler +1    /*     Reset handler */
       .word Default_Handler +1  /*       NMI handler */
       .word Default_Handler +1  /* HardFault handler */
       /* add rest of them here if needed */


/* reset handler */
.section .text
Reset_Handler:
       /* set stack pointer */
       ldr r0, =_estack
       mov sp, r0

       /* initialize data and bss
        * not necessary for rom only code
        * */
       bl init_data
       /* call main */
       bl main
       /* trap if returned */
       b .


/* initialize data and bss sections */
.section .text
init_data:

       /* copy rom to ram */
       ldr r0, =_sdata
       ldr r1, =_edata
       ldr r2, =_sidata
       movs r3, #0
       b LoopCopyDataInit

       CopyDataInit:
              ldr r4, [r2, r3]
              str r4, [r0, r3]
              adds r3, r3, #4

       LoopCopyDataInit:
```

```
              adds r4, r0, r3
              cmp r4, r1
              bcc CopyDataInit

       /* zero bss */
       ldr r2, =_sbss
       ldr r4, =_ebss
       movs r3, #0
       b LoopFillZerobss

       FillZerobss:
              str  r3, [r2]
              adds r2, r2, #4

       LoopFillZerobss:
              cmp r2, r4
              bcc FillZerobss

       bx lr


/* default handler */
.section .text
Default_Handler:
       b Default_Handler


/* main function */
.section .text
main:
       /* enable GPIOA and GPIOB clock, bit0 and bit1 on IOPENR */
       ldr r6, =RCC_IOPENR
       ldr r5, [r6]
       movs r4, 0x3
       orrs r5, r5, r4 /*Enable GPIOA and GPIOB clock*/
       str r5, [r6]/*Store the r5 register value to IOPENR register*/

       /* setup PA11 and PA12 for led 01 for bits 22-23 and 24-25 in MODER */
       ldr r6, =GPIOA_MODER
       ldr r5, [r6]
       ldr r4, =0x3C00000
       mvns r4, r4
       ands r5, r5, r4
       ldr r4, =0x1400000
       orrs r5, r5, r4
       str r5, [r6]

       /* turn on led connected to A11 and A12 in ODR */
       ldr r6, =GPIOA_ODR
       ldr r5, [r6]
       ldr r4,=0x1800
       orrs r5, r5, r4
       str r5, [r6]

       /* setup PB4 and PB5 for led 01 for bits 12-13 in MODER */
       ldr r6, =GPIOB_MODER
       ldr r5, [r6]
       ldr r4, =0xF00
       mvns r4, r4
```

```
        ands r5, r5, r4
        ldr r4, =0x500
        orrs r5, r5, r4
        str r5, [r6]

        /* turn on led connected to B4 and B5 in ODR */
        ldr r6, =GPIOB_ODR
        ldr r5, [r6]
        ldr r4,=0x30
        orrs r5, r5, r4
        str r5, [r6]


        /* for(;;); */
        b .

        /* this should never get executed */
        nop
```

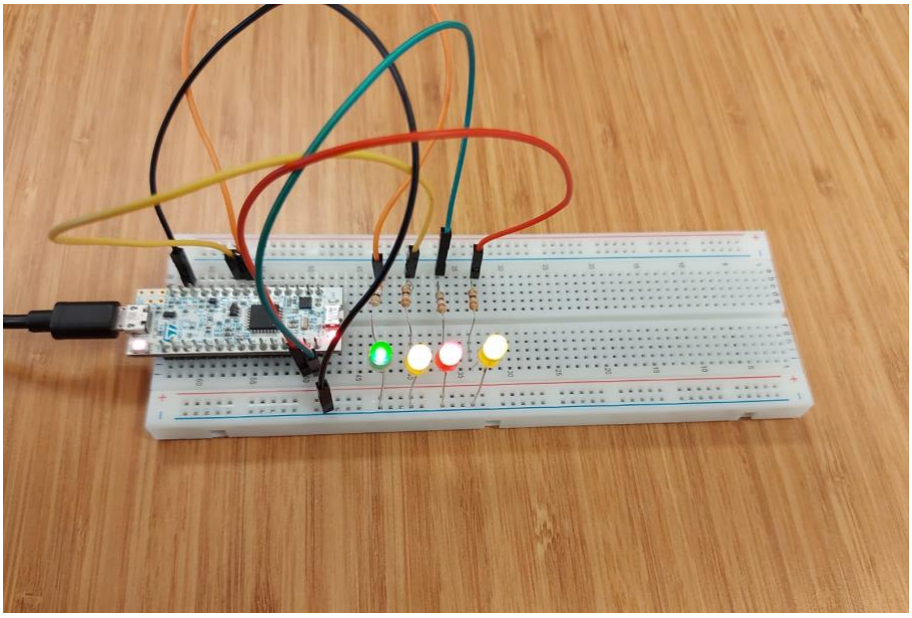


Figure 9: Four led on

5 Problem 4

5.1 Clock settings

In the beginning, the main function configures the microcontroller by enabling the clock for GPIOA (RCC_IOPENR register). To do this GPIOAEN is set to 1.

5.2 GPIO (PA8) configuration

The pin PA8 is set to output mode by using GPIOA_MODER register. To do this the Mode 8 bits is set to 01.

5.3 Toggling led and delay loop

Following the configuration, the ***blink_loop*** is initiated. This loop toggles the state of the LED on pin PA8 by reading the current state from the Output Data Register (GPIOA_ODR), XORing it with a mask

that represents the state of the LED and storing the result back to the ODR. This effectively toggles the LED. The mask is 0x100, it makes only the bit 8 (OD8 bit) toggle and don't effect to other bits.

After toggling the LED state, the program introduces a delay by executing a simple countdown loop (delay_loop). The **DELAY_FREQ** constant determines the duration of the delay, adjusting the speed of the LED blinking. **DELAY_FREQ** is loaded to r0 and subtract one with ***subs*** instruction, reference to Cortex-M0 Technical Reference Manual (figure 10) it will take one cycle than with ***bne*** instruction if r0 not equal to 0 they will go again to the countdown loop (delay_loop), reference to Cortex-M0 Technical Reference Manual it will take three cycle . The loop continues until the countdown reaches zero. Then go again to blink_loop label and toggle the led. So the delay_loop function execution time is 4 cycle (1 from subs, and 3 from ben). Assume the Clock is 16Mhz so 16M cycle appear in 1 second, so for waiting for 1 second we have to do a process of 16M cycle, for do this we made the loop count for 4M times, that's make 16M cycle. The code with **DELAY_FREQ** = 4Mhz was tried, and a clock was kept, the result of the observation was that the LED was delayed by less than 1 second, faster than expected. Therefore, dividing the frequency by 3 was tried and the code was run. When done this way, it was observed that the delay was 1 second. Although we cannot fully explain the reason for this, it seems that it may be because the loop execution time is just 3 cycles.

Subtract	Negate	`RSBS Rd, Rn, #0`	1
Branch	Conditional	`B <cc> <label>`	1 or 3[d]
	Unconditional	`B <label>`	3
	With link	`BL <label>`	4
	With exchange	`BX Rm`	3
	With link and exchange	`BLX Rm`	3

Figure 10: Time of instructions Cortex-M0 Technical Reference Manual

```
/*
 * asm.s
 *
 * author: Fouad Aladhami/Büşra Ülker/Ayşe Beri Çevik
 *
 * description: Added the necessary stuff for turning on the green LED on the
 *   G031K8 Nucleo board. Mostly for teaching.
 */



.syntax unified
.cpu cortex-m0plus
.fpu softvfp
.thumb



/* make linker see this */
.global Reset_Handler

/* get these from linker script */
.word _sdata
.word _edata
.word _sbss
.word _ebss



/* define peripheral addresses from RM0444 page 57, Tables 3-4 */
.equ RCC_BASE,         (0x40021000)          // RCC base address
.equ RCC_IOPENR,       (RCC_BASE   + (0x34)) // RCC IOPENR register offset
```

```
.equ GPIOA_BASE,       (0x50000000)         // GPIOA base address?
.equ GPIOA_MODER,      (GPIOA_BASE + (0x00)) // GPIOA MODER register
offset
.equ GPIOA_ODR,        (GPIOA_BASE + (0x14)) // GPIOA ODR register offset

.equ DELAY_FREQ,                    (16000000/3)

/* vector table, +1 thumb mode */
.section .vectors
vector_table:
        .word _estack             /*     Stack pointer */
        .word Reset_Handler +1    /*     Reset handler */
        .word Default_Handler +1  /*       NMI handler */
        .word Default_Handler +1  /* HardFault handler */
        /* add rest of them here if needed */


/* reset handler */
.section .text
Reset_Handler:
        /* set stack pointer */
        ldr r0, =_estack
        mov sp, r0

        /* initialize data and bss
         * not necessary for rom only code
         * */
        bl init_data
```

```
        /* call main */
        bl main
        /* trap if returned */
        b .


/* initialize data and bss sections */
.section .text
init_data:

        /* copy rom to ram */
        ldr r0, =_sdata
        ldr r1, =_edata
        ldr r2, =_sidata
        movs r3, #0
        b LoopCopyDataInit

        CopyDataInit:
                ldr r4, [r2, r3]
                str r4, [r0, r3]
                adds r3, r3, #4

        LoopCopyDataInit:
                adds r4, r0, r3
                cmp r4, r1
                bcc CopyDataInit

        /* zero bss */
        ldr r2, =_sbss
        ldr r4, =_ebss
```

```
        movs r3, #0
        b LoopFillZerobss

        FillZerobss:
                str  r3, [r2]
                adds r2, r2, #4

        LoopFillZerobss:
                cmp r2, r4
                bcc FillZerobss

        bx lr


/* default handler */
.section .text
Default_Handler:
        b Default_Handler


/* main function */
.section .text

main:
  /*Enable GPIOA clock, bit 0 on IOPENR*/
  ldr r6, =RCC_IOPENR
  ldr r5, [r6]
  movs r4, 0x1
  orrs r5, r5, r4
  str r5, [r6]
```

```
/* Set up PA8 for the LED (bits 16-17 in MODER)*/
ldr r6, =GPIOA_MODER
ldr r5, [r6]
ldr r4, =0x30000
bics r5, r5, r4 //erase bits 16 and 17
ldr r4, =0x10000
orrs r5, r5, r4 // write 01 to the bits
str r5, [r6]

/* Blink the LED*/
blink_loop:
        ldr r6, =GPIOA_ODR
  /*Read the current state of the LED*/
  ldr r5, [r6]
  /*Set r4 to the value 0x100 (bit 8)*/
  ldr r4, =0x100
  //Toggle the LED state by XORing with 0x100
  eors r5, r5, r4
        // Update the LED state
  str r5, [r6]

  // Delay between LED state changes (adjust for desired speed)
  ldr r0, =DELAY_FREQ
  delay_loop:
    subs r0, r0, #1 //This took 1 cycle
    bne delay_loop // This took 3 cycle

  // Repeat the blinking loop
  b blink_loop
```